Module:	CSC211 (2026)	Assessment:	Term 2 Practical 2
Lecturer:	Mr. C. K. Baker	Total:	50 marks
Duration:	Mon 12PM – Sat 12PM		

Topic
Binary Search Trees, File I/O, OOP, Error Handling and Testing

Instructions
There are 3 questions in this assignment. Submit a compressed (.zip) file with all your code. Use in-line Java comments to acknowledge sources consulted. The submission file should be named XXYYZZZ.zip where XXYYZZZ corresponds to your student number.

Question 1 – Create classes to define the BST [5 marks]

Study the class definitions below.

```
class Node {
    int data;
    Node left;
    Node right;
}

class BinarySearchTree {
    Node root;
    int size;
}
```

1.1 Copy the given code into two corresponding java files.

1.2 Use encapsulation to hide the attributes of `Node` and `BinarySearchTree`. Each attribute should only be visible in its own class.

1.3 Write code for the following:

- a default constructor for each class
- a loaded constructor for each class
- getter methods for the attributes in each class
- setter methods for the attributes in each class

Module:	CSC211 (2026)	Assessment:	Term 2 Practical 2
Lecturer:	Mr. C. K. Baker	Total:	50 marks
Duration:	Mon 12PM – Sat 12PM		

Question 2 – Implement methods to build and manipulate the BST [30 marks]

(EASY)

2.1 Implement the `boolean search(int data)` method

- return true if `data` exists, else return false

2.2 Implement the `void insert(int data)` method:

- insert a new node into the tree, maintaining the BST property
- if a duplicate is detected, print "Input not valid"
- update the size attribute after successful insertion

2.3 Implement the `int height()` method:

- calculate and return the height of the tree
- an empty tree has height 0

2.4 Implement the `void print_in_order()` method:

- print the in-order traversal of all elements in the BST

(MEDIUM)

2.5 Implement the `int find_kth_smallest(int k)` method

- if `k` is greater than the BST size, print out "Input not valid"
- perform an in-order traversal of the BST in-order
- keep track of the number of nodes visited
- when the kth node is reached, print out that node's value

(HARD)

2.6 Implement the `void delete(int data)` method:

- delete a node from the tree, maintaining the BST property
- if `data` does not exist, print "Input not valid"
- use the *in-order predecessor* for deletion of an internal node with two children
- update the size attribute after deletion

Module:	CSC211 (2026)	Assessment:	Term 2 Practical 2
Lecturer:	Mr. C. K. Baker	Total:	50 marks
Duration:	Mon 12PM – Sat 12PM		

Question 3 – Application [15 marks]

3.1 Create a file named `Main.java` in which to code your application.

3.2 Study the file below and implement the specifications that follow.

`input.txt`

```
BUILD 50 30 70 20 40 60 80
IN_ORDER
INSERT 65
INSERT 35
DELETE 30
IN_ORDER
SIZE
HEIGHT
KTH_SMALLEST 5
SEARCH 65
SEARCH 100
DELETE 10
```

3.2.1 Requirements

- create a new empty BST
- write code to read and process `input.txt` line by line
- each line contains a command followed by 0 or more arguments
- use the methods in 2.1 – 2.6 to execute the commands
- BUILD must use of the method `insert(int data)` in 2.2

3.2.2 Output rules

- print output after every command except BUILD, INSERT and DELETE
- format console output neatly in columns

3.2.3 Error handling and testing

- for invalid commands, incorrect arguments, or malformed input, print: "Input not valid"
- continue processing remaining commands until the end of the file is reached
- test your code on edge cases (boundary conditions) for each command
- test your code on different inputs for each command

Module:	CSC211 (2026)	Assessment:	Term 2 Practical 2
Lecturer:	Mr. C. K. Baker	Total:	50 marks
Duration:	Mon 12PM – Sat 12PM		

Example output (corresponding to `input.txt`)

```
IN_ORDER:          20 30 40 50 60 70 80
IN_ORDER:          20 35 40 50 60 65 70 80
SIZE:              8
HEIGHT:            3
KTH_SMALLEST:      60 (k = 5)
SEARCH 65:         true
SEARCH 100:        false
Input not valid
```

Total: 50 marks